

Most-general unifiers can have shallow images

Disproof of conjecture 00000006069

ziangni-sys

4 October 2026

Both versions of the source assert that the depth of a most-general unifier is bounded below by half the variable count. Three nontrivial equations on six distinct, actually occurring variables disprove this lower bound. The most-general substitution has only variable images and is idempotent after one application.

Actual first-order terms and constraints. Use variables $x_0, \dots, x_5$, a constant c and one unary function symbol F . Terms are recursively generated by these constructors. A substitution σ assigns a term to each variable and extends to all terms by

$$\widehat{\sigma}(x_i) = \sigma(x_i), \quad \widehat{\sigma}(c) = c, \quad \widehat{\sigma}(F(t)) = F(\widehat{\sigma}(t)).$$

Consider the finite equation set

$$E = \{x_0 = x_1, \quad x_2 = x_3, \quad x_4 = x_5\}.$$

All three equations have distinct sides. Every one of the six variables occurs in E , and there are no unused variables.

Define the simultaneous substitution

$$\theta(x_0) = x_1, \quad \theta(x_2) = x_3, \quad \theta(x_4) = x_5, \quad \theta(x_1) = x_1, \quad \theta(x_3) = x_3, \quad \theta(x_5) = x_5.$$

Both sides of each equation become its odd-indexed variable. Thus θ genuinely unifies all equations.

Universal most-generality, not just a certificate. Let τ be any substitution unifying E . Its images satisfy

$$\tau(x_0) = \tau(x_1), \quad \tau(x_2) = \tau(x_3), \quad \tau(x_4) = \tau(x_5).$$

For every variable, therefore, $\tau(x_i) = \widehat{\tau}(\theta(x_i))$. Induction on terms gives the full factorization

$$\widehat{\tau}(t) = \widehat{\tau}(\widehat{\theta}(t)) \quad \text{for every term } t.$$

Taking the residual substitution to be τ proves that every unifier is an instance of θ . Hence θ is a most-general unifier in the standard substitution ordering.

Depth under both usual leaf conventions. With zero-based term height, variables and constants have height zero and $F(t)$ has height $h(t)+1$. All six substitution images are variables, so their maximum height is zero. With one-based leaf height their maximum height is one. Thus

$$d_0(\theta) = 0, \quad d_1(\theta) = 1.$$

The actual variable count in the constraint set is six, so the claimed lower bound would require depth at least $6/2 = 3$. Both values are strictly smaller.

Even if variable count were interpreted as the number of variables changed by the substitution, the support is exactly $\{x_0, x_2, x_4\}$ and has cardinality three. The one-based depth 1 is still less than $3/2$, and zero-based depth is smaller still. Equivalently,

$$2d_1(\theta) = 2 < 3 < 6.$$

The substitution is also idempotent on all terms:

$$\widehat{\theta}(\widehat{\theta}(t)) = \widehat{\theta}(t).$$

There are no substitution chains of growing depth: every changed variable maps directly to a fixed variable in one step. This addresses a chain-depth interpretation as well.

Scope. The counterexample targets only the explicit variable-count depth lower bound. It does not require interpreting the undefined conversion constant, and makes no claim about complexity of a particular occurs-check implementation. The equations are nontrivial and all counted variables occur in them. Most-generality is proved for arbitrary substitutions and arbitrary terms, not asserted from a table of image values.

Formal correspondence. Lean defines an inductive first-order term type, recursive substitution application, recursive occurring-variable sets and term heights. The actual three-element equation set and substitution are finite objects on `Fin 6`. `occurring_count` and `nontrivial_equations` verify complete occurrence and nontriviality.

`theta_unifies` proves all constraints. `factor_variables` extracts the three equalities from an arbitrary unifier, and `theta_most_general` proves the universal factorization by term induction. `theta_idempotent` proves the full idempotence identity. The final `counterexample` combines actual most-generality, both depths, both variable counts and strict failures of the bound.

Reproduction. Run `lake build` in `lean/` with Lean 4.19.0 and public Mathlib revision

c44e0c8ee63ca166450922a373c7409c5d26b00b

The project prints the relevant axiom audits.